
Documentación Técnica

API de Reservas de Restaurantes

Proyecto de Bases de Datos II

Elemento	Detalle
Sistema documentado	API REST para gestión de restaurantes, menús, platos, usuarios, reservaciones, pedidos y búsqueda
Stack principal	Node.js, Express, PostgreSQL, MongoDB, Redis, Elasticsearch, Docker, GitHub Actions
Fuente revisada	Archivo BD_II-proyecto (10).zip
Estado de pruebas	8 suites aprobadas, 80 pruebas pasadas
Fecha	7 de mayo de 2026

Este documento resume y explica el diseño técnico del proyecto, su arquitectura, componentes de base de datos, endpoints principales, mecanismos de seguridad, pruebas automatizadas, despliegue con Docker y flujo de integración/entrega continua.

Índice

Índice	2
1. Resumen ejecutivo	5
2. Objetivos del sistema	5
2.1 Objetivo general	5
2.2 Objetivos específicos	5
3. Alcance funcional	5
4. Tecnologías utilizadas	6
5. Arquitectura general	6
5.1 Flujo general de una solicitud	7
6. Estructura del proyecto	7
6.1 Responsabilidad de carpetas importantes	8
7. Configuración y variables de entorno	8
8. Instalación y ejecución	9
8.1 Ejecución local sin Docker	9
8.2 Ejecución con PostgreSQL mediante Docker Compose	9
8.3 Ejecución con MongoDB simple	9
8.4 Ejecución con clúster MongoDB, Redis, Elasticsearch y Nginx	9
9. Modelo de datos en PostgreSQL	10
9.1 Relaciones principales	10
9.2 Inicialización SQL	10
10. Modelo de datos en MongoDB	10
10.1 Contadores secuenciales	11
10.2 Remoción de _id interno	11

11. Capa DAO y selección de motor	11
11.1 Ventajas del patrón utilizado	11
11.2 Funciones DAO principales	11
12. Seguridad y control de acceso	12
12.1 Registro de usuario	12
12.2 Login	12
12.3 Middlewares de autenticación	12
12.4 Permisos por módulo	12
13. Redis como caché	12
13.1 Funcionamiento del middleware	13
13.2 Invalidación	13
14. Elasticsearch y microservicio de búsqueda	13
14.1 Endpoints de búsqueda	13
14.2 Comportamiento tolerante a fallos	13
15. MongoDB sharding	13
15.1 Inicialización del clúster	14
15.2 Justificación de la shard key	14
15.3 Limitación actual	14
16. Documentación de endpoints	14
16.1 Ejemplos de solicitudes	15
17. Lógica por controlador	16
18. Pruebas automatizadas	16
18.1 Suites de prueba	16
18.2 Resultado verificado	17
18.3 Comandos de prueba	17

18.4 Pruebas con Docker	17
19. Integración continua y entrega continua	17
19.1 Flujo CI	17
19.2 Flujo CD	18
20. Docker, Nginx y despliegue	18
20.1 Nginx	18
21. Manejo de errores	18
22. Recomendaciones técnicas	18
23. Checklist de entrega	19
24. Conclusión	19
Anexo A. Comandos útiles	20
Anexo B. Resumen de rutas por archivo	20

1. Resumen ejecutivo

El proyecto corresponde a una API REST orientada a la administración de un sistema de reservas de restaurantes. La aplicación permite registrar usuarios, autenticar sesiones, administrar restaurantes, gestionar menús y platos, crear reservaciones, realizar pedidos y consultar información mediante un microservicio de búsqueda basado en Elasticsearch.

La implementación está construida con Node.js y Express, separando responsabilidades en rutas, controladores, modelos, middlewares, servicios y una capa DAO. Esta última es uno de los puntos más importantes del proyecto porque permite trabajar con PostgreSQL o MongoDB mediante la misma lógica de controladores, cambiando únicamente la variable de entorno que define el tipo de base de datos.

Además de la API principal, el sistema incorpora Redis como capa de caché para consultas frecuentes, Elasticsearch para búsquedas textuales, Docker para levantar los servicios, un clúster MongoDB con sharding para escenarios de distribución, y GitHub Actions para validar pruebas, construcción y publicación de imagen Docker.

El estado actual del proyecto fue verificado ejecutando la batería de pruebas automatizadas. El resultado fue positivo: 8 suites aprobadas y 80 pruebas pasadas. Esto deja el proyecto en una condición adecuada para entrega, siempre que se mantengan configuradas correctamente las variables de entorno y los servicios auxiliares.

2. Objetivos del sistema

2.1 Objetivo general

Desarrollar una API REST para la gestión integral de restaurantes, usuarios, menús, platos, reservaciones y pedidos, integrando bases de datos relacionales y documentales junto con tecnologías de caché, búsqueda, contenedorización y automatización de pruebas.

2.2 Objetivos específicos

- Implementar autenticación segura mediante registro, login, contraseñas cifradas y tokens JWT.
- Administrar restaurantes, menús y platos con validaciones de permisos según el rol del usuario.
- Permitir reservaciones y pedidos asociados a usuarios y restaurantes.
- Separar la lógica de acceso a datos mediante DAOs para soportar PostgreSQL y MongoDB.
- Usar Redis para reducir consultas repetidas en endpoints públicos de lectura.
- Incorporar Elasticsearch para búsquedas flexibles sobre restaurantes, menús y platos.
- Preparar despliegue mediante Docker, Docker Compose y Nginx.
- Configurar pruebas automatizadas y flujos CI/CD con GitHub Actions.

3. Alcance funcional

El alcance funcional del sistema se centra en operaciones de backend. No se documenta una interfaz gráfica como parte principal del proyecto; el consumo esperado es mediante clientes HTTP como Postman, Insomnia, una aplicación frontend externa o pruebas automatizadas.

Módulo	Responsabilidad principal
Autenticación	Registro de clientes y administradores, login y emisión de JWT.
Usuarios	Consulta de perfil, consulta administrativa, actualización y eliminación.

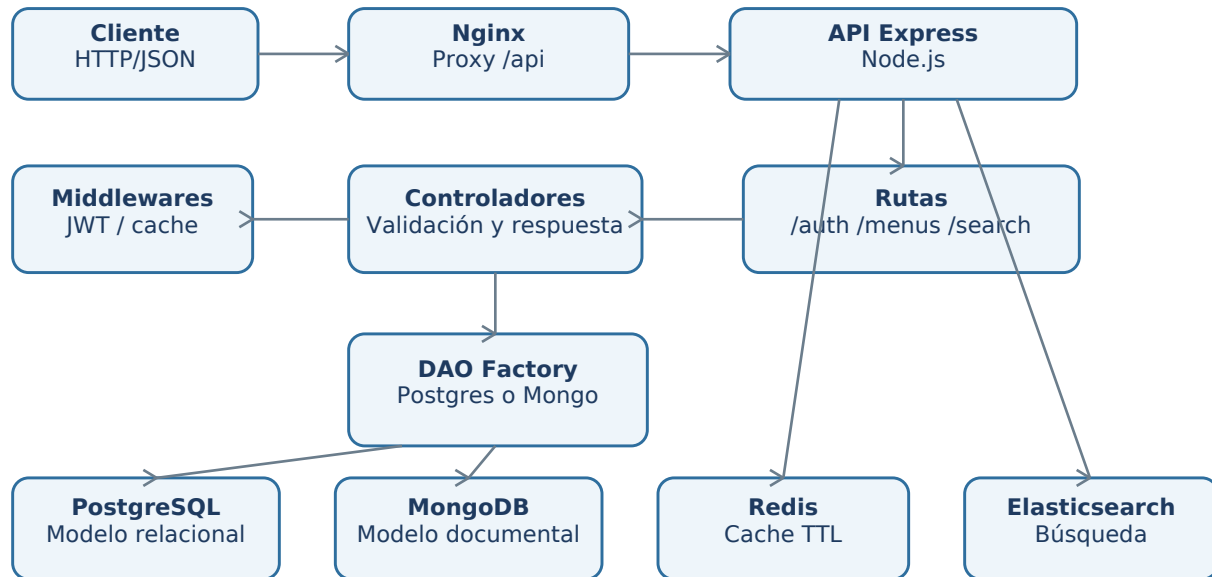
Módulo	Responsabilidad principal
Restaurantes	Listado público y creación controlada por clave de configuración.
Menús	Consulta pública y administración por el encargado del restaurante.
Platos	Consulta pública y administración dentro de un menú específico.
Reservaciones	Creación y cancelación de reservaciones por usuarios autenticados.
Pedidos	Creación de órdenes con ítems y cálculo de total.
Búsqueda	Consulta textual y reindexación de datos en Elasticsearch.
Infraestructura	PostgreSQL, MongoDB, Redis, Elasticsearch, Nginx, Docker y CI/CD.

4. Tecnologías utilizadas

Tecnología	Uso dentro del proyecto
Node.js	Entorno de ejecución del backend.
Express	Framework HTTP para rutas, middlewares y controladores.
PostgreSQL	Base de datos relacional principal para el esquema SQL.
MongoDB	Base de datos documental alternativa mediante DAOs Mongo.
Redis	Capa de caché con TTL para respuestas frecuentes.
Elasticsearch	Motor de búsqueda para restaurantes, menús y platos.
JWT	Autenticación basada en tokens Bearer.
bcrypt	Hash de contraseñas y códigos administrativos.
Docker	Construcción y ejecución de la aplicación en contenedores.
Docker Compose	Orquestación local de API, bases de datos y servicios auxiliares.
Nginx	Proxy inverso hacia la API.
Jest + Supertest	Pruebas automatizadas de endpoints y lógica HTTP.
GitHub Actions	Automatización de CI y publicación de imagen Docker.

5. Arquitectura general

La arquitectura está organizada en capas. El cliente se comunica con la API por HTTP/JSON. Las rutas reciben la solicitud, los middlewares verifican autenticación o aplican caché, los controladores validan reglas de negocio y la capa DAO accede a la base de datos configurada. Para lectura frecuente se usa Redis, mientras que para búsquedas textuales se consulta Elasticsearch.



5.1 Flujo general de una solicitud

- 1 El cliente envía una solicitud HTTP a un endpoint, por ejemplo GET /menus o POST /orders.
- 2 Express dirige la solicitud hacia la ruta correspondiente definida en src/routes.
- 3 Si el endpoint requiere seguridad, authenticateToken valida el token JWT enviado en Authorization: Bearer .
- 4 El controlador ejecuta validaciones de entrada, permisos y reglas de negocio.
- 5 La capa DAO seleccionada por src/daos/factory.js consulta PostgreSQL o MongoDB según DATABASE_TYPE.
- 6 Si aplica, Redis guarda o invalida información en caché.
- 7 Si aplica, Elasticsearch indexa, elimina o busca documentos.
- 8 La API responde en formato JSON con un mensaje, datos o error controlado.

6. Estructura del proyecto

La estructura separa el código principal dentro de restaurant-reservations. También existe una carpeta superior Tests con un Dockerfile de pruebas y archivos de prueba usados para ejecución aislada. El código activo de la aplicación está en src y las pruebas usadas por npm test están en restaurant-reservations/tests.

```

BD_II-proyecto/
├── .github/workflows/
│   ├── ci.yml
│   └── cd.yml
├── Tests/
│   ├── Dockerfile.test
│   ├── auth.test.js
│   ├── menus.test.js
│   ├── orders.test.js
│   ├── reservations.test.js
│   └── restaurants.test.js
└── restaurant-reservations/

```

```

├── Dockerfile
├── docker-compose.yml
├── docker-compose-mongo.yml
├── docker-compose-mongo-cluster.yml
├── jest.config.js
├── package.json
├── nginx/nginx.conf
├── scripts/init-mongo-cluster.ps1
├── src/
│   ├── app.js / server.js
│   ├── config/
│   ├── controllers/
│   ├── daos/postgres/
│   ├── daos/mongodb/
│   ├── middlewares/
│   ├── models/
│   ├── routes/
│   ├── services/
│   └── utils/
├── tests/
│   ├── auth.test.js
│   ├── users.test.js
│   ├── restaurants.test.js
│   ├── menus.test.js
│   ├── dishes.test.js
│   ├── reservations.test.js
│   ├── orders.test.js
│   └── search.test.js

```

6.1 Responsabilidad de carpetas importantes

Ruta	Contenido / función
src/app.js	Configura Express, JSON middleware, endpoint raíz y monta todas las rutas.
src/server.js	Inicializa base de datos según DATABASE_TYPE y levanta el servidor.
src/config	Conexiones a PostgreSQL, MongoDB, Elasticsearch e inicialización SQL/Mongo.
src/routes	Define endpoints públicos y protegidos.
src/controllers	Contiene la lógica HTTP, validaciones y respuestas.
src/daos/factory.js	Selecciona DAOs de PostgreSQL o MongoDB según variable de entorno.
src/daos/postgres	Implementación relacional usando pg y consultas SQL.
src/daos/mongodb	Implementación documental usando el driver oficial de MongoDB.
src/middlewares	Autenticación JWT, autorización por rol y caché.
src/services/searchService.js	Creación de índice, búsqueda, reindexación e integración con Elasticsearch.
src/utils/cache.js	Cliente Redis y funciones get, set y del.
tests	Pruebas automatizadas con Jest y Supertest.
.github/workflows	Pipelines de CI y CD.

7. Configuración y variables de entorno

La aplicación utiliza dotenv para cargar variables desde el archivo .env. En producción o CI estas variables deberían definirse desde el entorno y no quemarse directamente en el código. En ningún caso se recomienda subir secretos reales al repositorio.

Variable	Descripción	Ejemplo / valor esperado
PORT	Puerto donde escucha la API.	3000

Variable	Descripción	Ejemplo / valor esperado
DATABASE_TYPE	Motor de base de datos activo.	postgres, postgresql, pg, mongodb o mongo
DB_HOST	Host de PostgreSQL.	db o localhost
DB_PORT	Puerto de PostgreSQL.	5432
DB_USER	Usuario de PostgreSQL.	postgres
DB_PASSWORD	Contraseña de PostgreSQL.	valor seguro
DB_NAME	Nombre de la base de datos.	restaurant_db
MONGO_URI	Cadena de conexión de MongoDB.	mongodb://mongo:27017 o mongodb://mongos:27017
MONGO_DB_NAME	Nombre de base en MongoDB.	restaurant_db
JWT_SECRET	Clave para firmar tokens JWT.	secreto fuerte
RESTAURANT_SETUP_KEY	Clave para permitir creación inicial de restaurantes.	valor secreto
REDIS_URL	URL de conexión a Redis.	redis://redis:6379
ELASTICSEARCH_URL	URL de Elasticsearch.	http://elasticsearch:9200
ELASTICSEARCH_ENABLE	Permite desactivar Elasticsearch si vale false.	true/false

8. Instalación y ejecución

8.1 Ejecución local sin Docker

Este flujo requiere tener Node.js instalado y contar con una instancia de PostgreSQL, MongoDB, Redis o Elasticsearch según el modo que se quiera probar. Para una ejecución mínima basta con la API y una base de datos compatible.

```
cd restaurant-reservations
npm install
npm start
```

Para desarrollo con reinicio automático se puede usar:

```
npm run dev
```

8.2 Ejecución con PostgreSQL mediante Docker Compose

El archivo docker-compose.yml levanta dos servicios: api y db. La base usa PostgreSQL 15 Alpine y carga el script src/config/init.sql al inicializarse.

```
cd restaurant-reservations
docker compose up --build
```

8.3 Ejecución con MongoDB simple

El archivo docker-compose-mongo.yml levanta la API usando DATABASE_TYPE=mongodb y un contenedor MongoDB único. Es útil para validar el modo documental sin configurar un clúster completo.

```
cd restaurant-reservations
docker compose -f docker-compose-mongo.yml up --build
```

8.4 Ejecución con clúster MongoDB, Redis, Elasticsearch y Nginx

El archivo docker-compose-mongo-cluster.yml levanta la API conectada a mongos, Redis, Elasticsearch, Nginx, tres config servers y tres nodos del shard 1. Después de levantar los contenedores, el script

PowerShell inicializa replica sets y activa sharding.

```
cd restaurant-reservations
docker compose -f docker-compose.mongo-cluster.yml up --build
powershell -ExecutionPolicy Bypass -File scripts/init-mongo-cluster.ps1
```

9. Modelo de datos en PostgreSQL

El modelo relacional se crea desde `src/config/init.sql`. Las tablas usan claves primarias SERIAL, relaciones mediante claves foráneas y marcas `created_at/updated_at` en varias entidades. Esta versión es útil para mantener integridad referencial fuerte, especialmente en usuarios, restaurantes, menús, platos, reservaciones y pedidos.

Tabla	Campos principales	Relaciones / observaciones
restaurants	id, name, admin_code_hash, created_at	name es único. admin_code_hash permite registrar administradores mediante código seguro.
users	id, name, email, password_hash, role, restaurant_id, created_at, updated_at	email único. restaurant_id referencia restaurants. role controla permisos client/admin.
menus	id, name, description, restaurant_id, created_at, updated_at	restaurant_id referencia restaurants con borrado en cascada.
dishes	id, name, description, price, menu_id, created_at, updated_at	menu_id referencia menus. price usa DECIMAL(10,2).
reservations	id, user_id, restaurant_id, reservation_date, reservation_time, num_people, status	Asocia usuarios y restaurantes. status inicia como pending.
orders	id, user_id, restaurant_id, order_type, status, total, created_at, updated_at	Controla pedidos pickup/dine_in y total acumulado.
order_items	id, order_id, dish_id, quantity, unit_price, subtotal	Detalle del pedido. dish_id usa ON DELETE RESTRICT para proteger historial.

9.1 Relaciones principales

- Un restaurante puede tener muchos menús.
- Un menú puede tener muchos platos.
- Un usuario puede crear muchas reservaciones y muchos pedidos.
- Un restaurante puede recibir muchas reservaciones y pedidos.
- Un pedido se compone de varios order_items, cada uno asociado a un plato.
- Un usuario administrador puede estar asociado a un restaurante mediante restaurant_id.

9.2 Inicialización SQL

En modo PostgreSQL, `server.js` ejecuta `initDb` antes de levantar el servidor. El contenedor PostgreSQL también recibe `init.sql` en `/docker-entrypoint-initdb.d/init.sql`, por lo que el esquema puede inicializarse automáticamente al crear el volumen por primera vez.

10. Modelo de datos en MongoDB

La implementación en MongoDB replica la funcionalidad de los DAOs de PostgreSQL, pero trabajando con colecciones. Para mantener compatibilidad con los controladores, los documentos conservan campos como `id`, `restaurant_id`, `menu_id` y `user_id`. Esto permite que la capa superior no dependa directamente del motor de base de datos.

Colección	Índices definidos	Uso
restaurants	id único, name único	Almacena locales y hash de código de administrador.
users	id único, email único, restaurant_id	Almacena usuarios, roles y relación con restaurante.
menus	id único, restaurant_id	Permite listar menús por restaurante.
dishes	id, menu_id, restaurant_id, restaurant_id + menu_id	Optimiza consultas de platos por restaurante y menú.
reservations	id, user_id, restaurant_id, restaurant_id + user_id	Optimiza búsqueda de reservaciones por usuario/restaurante.
orders	id único, user_id, restaurant_id, restaurant_id + user_id	Soporta pedidos y consultas por usuario/restaurante.

10.1 Contadores secuenciales

MongoDB no genera SERIAL como PostgreSQL. Por eso los DAOs Mongo utilizan una colección de counters mediante la función getNextId. Cada entidad puede obtener un id incremental compatible con la estructura usada por los controladores y las pruebas.

10.2 Remoción de _id interno

Los DAOs Mongo usan utilidades como removeMongold para evitar exponer el campo interno _id en las respuestas de la API. Esto ayuda a mantener un contrato JSON similar al de PostgreSQL.

11. Capa DAO y selección de motor

La capa DAO es una de las decisiones de diseño más importantes. Los controladores no llaman directamente a PostgreSQL ni a MongoDB. En su lugar importan objetos como userDao, restaurantDao, menuDao, dishDao, reservationDao y orderDao desde src/daos/factory.js.

```
const dbType = (process.env.DATABASE_TYPE || "postgres").toLowerCase();

if ([ "postgres", "postgresql", "pg" ].includes(dbType)) {
  module.exports = { userDao, restaurantDao, menuDao, dishDao, reservationDao, orderDao };
} else if ([ "mongodb", "mongo" ].includes(dbType)) {
  module.exports = { userDao, restaurantDao, menuDao, dishDao, reservationDao, orderDao };
}
```

11.1 Ventajas del patrón utilizado

- Permite cambiar de PostgreSQL a MongoDB con una variable de entorno.
- Evita duplicar la lógica de controladores.
- Facilita pruebas unitarias porque los DAOs pueden mockearse.
- Reduce acoplamiento entre reglas de negocio y tecnología de persistencia.
- Hace visible la comparación entre modelo relacional y modelo documental, útil para Bases de Datos II.

11.2 Funciones DAO principales

DAO	Funciones destacadas
userDao	create, findByEmail, findById, update, deleteUser, findAll, findAdminByRestaurantId
restaurantDao	findRestaurantByName, getAllRestaurants, createRestaurant, findRestaurantById
menuDao	getAllMenus, getMenuById, createMenu, updateMenu, deleteMenu
dishDao	getDishesByMenuId, getDishById, createDish, updateDish, deleteDish

DAO	Funciones destacadas
reservationDao	createReservation, cancelReservation, getReservationById
orderDao	getOrderById, getOrderItems, createOrder, addOrderItem, updateOrderTotal, updateOrderStatus

12. Seguridad y control de acceso

La seguridad se implementa principalmente mediante JWT, bcrypt y validaciones de rol. Las contraseñas de usuarios no se almacenan en texto plano; se guardan como password_hash. De forma similar, el código administrativo de cada restaurante se guarda como admin_code_hash.

12.1 Registro de usuario

El endpoint POST /auth/register recibe nombre, email, password y opcionalmente role. Si el usuario se registra como cliente, el rol final será client. Si intenta registrarse como admin, debe enviar restaurantName y restaurantCode. El sistema valida que el restaurante exista y que el código coincida con el hash guardado.

12.2 Login

El endpoint POST /auth/login valida email y password. Si las credenciales son correctas, genera un token JWT con id, email, role y restaurantId. Este token debe enviarse en endpoints protegidos usando el encabezado Authorization.

Authorization: Bearer <token_jwt>

12.3 Middlewares de autenticación

Middleware	Función
authenticateToken	Exige un token JWT válido. Si falta, responde 401. Si es inválido o expiró, responde 403.
optionalAuth	Verifica el token si existe, pero permite continuar sin usuario autenticado.
authorizeRole	Permite limitar endpoints a roles específicos, por ejemplo admin.

12.4 Permisos por módulo

Recurso	Lectura	Escritura / cambios
Restaurantes	Pública	Creación controlada por optionalAuth y clave de configuración.
Menús	Pública con caché	Solo administrador autenticado del restaurante.
Platos	Pública con caché	Solo administrador autenticado del restaurante del menú.
Reservaciones	Requiere usuario autenticado	Usuario autenticado.
Pedidos	Requiere usuario autenticado	Usuario autenticado.
Usuarios	Perfil propio o admin	Propio usuario o admin según ruta.
Reindexación búsqueda	No aplica	Solo admin autenticado.

13. Redis como caché

Redis se usa para guardar respuestas de endpoints de lectura frecuente por un tiempo definido. El middleware cacheMiddleware crea una llave con req.originalUrl, revisa si ya existe una respuesta y, si existe, la devuelve sin consultar nuevamente el controlador o la base de datos.

13.1 Funcionamiento del middleware

- 1 Construye una clave con el formato cache:.
- 2 Consulta Redis usando cache.get(key).
- 3 Si existe contenido, responde inmediatamente con JSON.
- 4 Si no existe, intercepta res.json para guardar la respuesta antes de enviarla.
- 5 Guarda la respuesta con setEx y TTL, por defecto 60 segundos.

13.2 Invalidación

Cuando se crean, actualizan o eliminan menús o platos, los controladores llaman a cache.del con patrones como cache:/menus* o cache:/menus/*/dishes*. Esto evita que el cliente reciba información vieja después de una modificación.

14. Elasticsearch y microservicio de búsqueda

El módulo de búsqueda permite consultar restaurantes, menús y platos desde un índice común. El servicio searchService.js crea el índice si no existe, define mapeos y usa multi_match para buscar por nombre, descripción y nombre de restaurante. También soporta filtros por tipo y restaurantId.

Campo indexado	Uso
type	Distingue restaurant, menu o dish.
entity_id	Identificador original de la entidad.
name	Campo textual con mayor peso en la búsqueda.
description	Campo textual con peso intermedio.
restaurant_id	Filtro y relación con restaurante.
restaurant_name	Permite encontrar resultados por nombre del local.
menu_id	Relación de platos con menú.
price	Precio de platos cuando aplica.
indexed_at	Fecha de indexación.

14.1 Endpoints de búsqueda

Método	Ruta	Autenticación	Descripción
GET	/search?q=&type;=&restaurantId;=&limit;=	Pública	Busca documentos en Elasticsearch. Si Elasticsearch no está disponible, responde available=false y lista vacía.
POST	/search/reindex	Admin	Reconstruye el índice recorriendo restaurantes, menús y platos desde los DAOs.

14.2 Comportamiento tolerante a fallos

El cliente de Elasticsearch revisa disponibilidad mediante ping. Si el servicio no responde, no se detiene toda la API; el módulo de búsqueda devuelve available=false o ignora indexaciones. Esto permite ejecutar pruebas y módulos principales aun sin Elasticsearch activo.

15. MongoDB sharding

El proyecto incluye una configuración de clúster MongoDB con sharding. Esta parte es importante porque demuestra distribución de datos más allá de una base documental simple. El compose crea tres config servers, tres nodos del shard 1 y un router mongos.

Componente	Cantidad	Responsabilidad
Config servers	3	Guardan metadatos del clúster sharded.
Shard 1	3 nodos	Replica set que almacena datos particionados.
mongos	1	Router que recibe conexiones de la API y dirige operaciones al shard.
Redis	1	Caché compartida.
Elasticsearch	1	Búsqueda textual.
Nginx	1	Proxy hacia la API.

15.1 Inicialización del clúster

El script `scripts/init-mongo-cluster.ps1` ejecuta los pasos necesarios mediante `docker exec` y `mongosh`:

- 1 Inicia el replica set de config servers con id `cfgrs`.
- 2 Inicia el replica set `shard1rs` con `shard1a`, `shard1b` y `shard1c`.
- 3 Agrega el shard al router mongos.
- 4 Activa sharding en la base `restaurant_db`.
- 5 Configura sharding para `restaurant_db.dishes` con clave `restaurant_id` hashed.
- 6 Configura sharding para `restaurant_db.reservations` con clave `restaurant_id` hashed.

15.2 Justificación de la shard key

La clave `restaurant_id` hashed reparte datos por restaurante evitando que un solo restaurante concentre todas las operaciones en un mismo rango. Es adecuada para colecciones como `dishes` y `reservations` porque ambas se consultan normalmente por restaurante y pueden crecer con el tiempo.

15.3 Limitación actual

El clúster contiene un único shard compuesto por tres nodos. Esto demuestra réplica y configuración de sharding, pero para observar distribución real entre varios shards sería necesario agregar un segundo shard replica set y volver a balancear colecciones.

16. Documentación de endpoints

Los endpoints devuelven respuestas JSON. Los endpoints protegidos requieren enviar `Authorization: Bearer`. Las validaciones devuelven códigos 400, 401, 403, 404 o 409 según el caso, y errores inesperados devuelven 500.

Método	Ruta	Auth	Descripción
GET	/	No	Verifica que la API esté funcionando. Responde mensaje de bienvenida.
POST	/auth/register	No	Registra usuario cliente o administrador si aporta datos del restaurante y código.
POST	/auth/login	No	Autentica usuario y devuelve token JWT.
GET	/restaurants	No	Lista restaurantes.

Método	Ruta	Auth	Descripción
POST	/restaurants	Opcional	Crea restaurante usando clave de configuración o reglas del controlador.
GET	/users/me	Sí	Consulta perfil del usuario autenticado.
GET	/users	Admin	Lista usuarios.
GET	/users/:id	Admin	Obtiene usuario por id.
PUT	/users/:id	Sí	Actualiza usuario.
DELETE	/users/:id	Admin	Elimina usuario.
GET	/menus	No	Lista menús; acepta restaurantId como query param.
GET	/menus/:id	No	Obtiene menú por id.
POST	/menus	Admin	Crea menú para el restaurante del administrador.
PUT	/menus/:id	Admin	Actualiza menú si pertenece al restaurante del admin.
DELETE	/menus/:id	Admin	Elimina menú si pertenece al restaurante del admin.
GET	/menus/:menuId/dishes	No	Lista platos de un menú.
GET	/menus/:menuId/dishes/:id	No	Obtiene plato por id.
POST	/menus/:menuId/dishes	Admin	Crea plato dentro de un menú del restaurante del admin.
PUT	/menus/:menuId/dishes/:id	Admin	Actualiza plato.
DELETE	/menus/:menuId/dishes/:id	Admin	Elimina plato.
POST	/reservations	Sí	Crea reservación.
DELETE	/reservations/:id	Sí	Cancela reservación.
POST	/orders	Sí	Crea pedido con ítems.
GET	/orders/:id	Sí	Consulta detalle del pedido.
GET	/search	No	Realiza búsqueda textual.
POST	/search/reindex	Admin	Reconstruye el índice de búsqueda.

16.1 Ejemplos de solicitudes

Registro de cliente:

POST /auth/register
Content-Type: application/json

```
{
  "name": "Cliente Demo",
  "email": "cliente@demo.com",
  "password": "password123"
}
```

Registro de administrador de restaurante:

POST /auth/register
Content-Type: application/json

```
{
  "name": "Admin Restaurante",
  "email": "admin@demo.com",
  "password": "password123",
  "role": "admin",
  "restaurantName": "Restaurante Central",
  "restaurantCode": "codigo-del-local"
}
```

Crear menú como administrador:

```
POST /menus
Authorization: Bearer <token_admin>
Content-Type: application/json

{
  "name": "Menú de almuerzo",
  "description": "Opciones disponibles al mediodía",
  "restaurantId": 1
}
```

Crear pedido:

```
POST /orders
Authorization: Bearer <token_cliente>
Content-Type: application/json

{
  "restaurantId": 1,
  "orderType": "pickup",
  "items": [
    { "dishId": 1, "quantity": 2 },
    { "dishId": 3, "quantity": 1 }
  ]
}
```

Búsqueda textual:

```
GET /search?q=pizza&type=dish&restaurantId=1&limit=10
```

17. Lógica por controlador

Controlador	Responsabilidades principales
authController	Normaliza datos, valida contraseña, maneja registro de client/admin, compara hashes con bcrypt y genera JWT.
restaurantController	Lista restaurantes y controla la creación de nuevos locales con código administrativo.
UserController	Obtiene perfil, usuarios por id, lista de usuarios, actualización y eliminación con validación de permisos.
menuController	Lista, consulta, crea, actualiza y elimina menús. Valida que el admin pertenezca al restaurante.
dishController	Administra platos anidados bajo menús, valida existencia del menú e invalida caché.
reservationController	Crea reservaciones y permite cancelar reservaciones existentes.
orderController	Crea pedidos, agrega ítems, calcula totales y devuelve detalles.
searchController	Ejecuta búsqueda y reindexación usando searchService.

18. Pruebas automatizadas

El proyecto cuenta con pruebas automatizadas mediante Jest y Supertest. Las pruebas validan rutas principales y casos de error sin depender necesariamente de una base real, ya que se utilizan mocks para DAOs, cache y búsqueda. Esto permite verificar la lógica HTTP, validaciones, permisos y respuestas esperadas.

18.1 Suites de prueba

Suite	Enfoque
auth.test.js	Registro, login, validación de credenciales y roles.
users.test.js	Perfil, consultas de usuario, actualización y eliminación.

Suite	Enfoque
restaurants.test.js	Listado y creación de restaurantes.
menus.test.js	Listado, consulta, creación, actualización y eliminación de menús.
dishes.test.js	Platos por menú y control de permisos.
reservations.test.js	Creación y cancelación de reservaciones.
orders.test.js	Creación y consulta de pedidos.
search.test.js	Búsqueda y reindexación.

18.2 Resultado verificado

La ejecución local de pruebas produjo el siguiente resultado:

```
Test Suites: 8 passed, 8 total
Tests:      80 passed, 80 total
Snapshots:  0 total
Time:       20.5 s
Ran all test suites.
```

Durante la ejecución se observó una advertencia de Jest relacionada con handles abiertos, por lo que se usó `--forceExit` para cerrar el proceso al finalizar. Esto no invalida las pruebas, pero sí es una recomendación técnica revisar conexiones abiertas de Redis, MongoDB o Elasticsearch en futuros ajustes.

18.3 Comandos de prueba

```
cd restaurant-reservations
npm test
npm test -- --runInBand
npm test -- --runInBand --forceExit
```

18.4 Pruebas con Docker

La carpeta Tests incluye un Dockerfile.test que instala dependencias, copia código y pruebas, y permite ejecutar Jest dentro de un contenedor. Esto es útil para validar en un ambiente aislado.

```
docker build -f Tests/Dockerfile.test -t restaurant-api:tests .
docker run --rm restaurant-api:tests
```

19. Integración continua y entrega continua

El proyecto usa GitHub Actions con dos workflows: CI - Restaurant Reservations API y CD - Publish Docker Image. El primero valida calidad y construcción; el segundo publica una imagen Docker en GitHub Container Registry si CI termina correctamente.

19.1 Flujo CI

Paso	Descripción
Checkout	Descarga el código del repositorio.
Setup Node.js	Configura Node.js 20 y cache de npm.
npm install	Instala dependencias del proyecto.
npm test -- --runInBand	Ejecuta pruebas automatizadas.
Carga con PostgreSQL	Valida que src/app.js cargue en modo PostgreSQL.
Carga con MongoDB	Valida que src/app.js cargue con DATABASE_TYPE=mongodb.
node -c src/server.js	Comprueba sintaxis del archivo principal del servidor.

Paso	Descripción
docker build	Valida que la imagen Docker pueda construirse.

19.2 Flujo CD

El workflow de CD se activa cuando el workflow de CI finaliza exitosamente en main o master. Luego inicia sesión en GHCR, configura Docker Buildx y publica la imagen con dos etiquetas: latest y el hash del commit.

```
ghcr.io/<owner>/restaurant-reservations-api:latest
ghcr.io/<owner>/restaurant-reservations-api:<sha-del-commit>
```

20. Docker, Nginx y despliegue

El Dockerfile principal usa node:20-alpine, copia package.json, instala dependencias de producción, copia el proyecto, expone el puerto 3000 y ejecuta node src/server.js.

Archivo	Propósito
Dockerfile	Construcción de la imagen de la API.
docker-compose.yml	API + PostgreSQL.
docker-compose-mongo.yml	API + MongoDB simple.
docker-compose-mongo-cluster.yml	API + mongos + config servers + shard + Redis + Elasticsearch + Nginx.
nginx/nginx.conf	Proxy para /api, /search y endpoint /health.

20.1 Nginx

Nginx se configura como proxy inverso. Las solicitudes a /api/ se redirigen hacia el backend api:3000, /search/ también apunta a la API y /health responde OK directamente desde Nginx.

21. Manejo de errores

Los controladores usan bloques try/catch para responder con error interno del servidor cuando ocurre una excepción no controlada. Además, se manejan errores de validación comunes antes de llegar a la base de datos.

Código HTTP	Uso típico en el proyecto
200	Operación exitosa de consulta o actualización.
201	Creación exitosa de usuario, menú, plato, reservación, pedido o restaurante.
400	Faltan campos obligatorios o no hay campos para actualizar.
401	Credenciales inválidas o token no proporcionado.
403	Token inválido/expirado o usuario sin permisos suficientes.
404	Entidad no encontrada: usuario, restaurante, menú, plato, etc.
409	Conflicto de datos, por ejemplo restaurante ya tiene administrador.
500	Error interno no esperado.

22. Recomendaciones técnicas

- No subir archivos .env con secretos reales al repositorio. Usar variables de entorno o secrets de GitHub Actions.

- Cerrar conexiones abiertas en pruebas para evitar la advertencia de Jest sobre handles abiertos.
- Agregar más shards si se desea demostrar distribución real de datos en MongoDB, no solo configuración de sharding.
- Documentar ejemplos de Postman o colecciones HTTP para facilitar la revisión del profesor.
- Agregar validaciones más estrictas de formatos de fecha, hora y precios.
- Usar refresh controlado en Elasticsearch en producción para no afectar rendimiento.
- Evitar cachear respuestas privadas de usuarios si en el futuro se agregan endpoints con información sensible.

23. Checklist de entrega

Elemento	Estado sugerido
API Express funcional	Completado
PostgreSQL con init.sql	Completado
MongoDB con DAOs equivalentes	Completado
Redis como caché	Completado
Elasticsearch para búsqueda	Completado
MongoDB sharding	Completado como configuración base con un shard replica set
Docker y Docker Compose	Completado
Nginx como proxy	Completado
Pruebas automatizadas	Completado: 8 suites, 80 pruebas
CI con GitHub Actions	Completado
CD publicando imagen Docker	Completado
Documentación técnica	Completado en este PDF

24. Conclusión

El sistema presenta una arquitectura backend completa para una API de reservas de restaurantes. El uso de una capa DAO permite comparar PostgreSQL y MongoDB sin reescribir controladores, lo cual aporta valor académico para un curso de Bases de Datos II. Además, la integración de Redis, Elasticsearch, Docker, MongoDB sharding y GitHub Actions demuestra un enfoque más amplio que una API CRUD básica.

Con las pruebas automatizadas aprobadas y la documentación técnica finalizada, el proyecto queda listo para ser presentado como una solución funcional, modular y extensible. Las principales mejoras futuras se concentran en endurecer seguridad, cerrar conexiones en pruebas, ampliar el clúster sharded y complementar la entrega con colecciones de Postman o ejemplos de consumo desde un cliente frontend.

Anexo A. Comandos útiles

```
# Instalar dependencias
cd restaurant-reservations
npm install

# Ejecutar API
npm start

# Ejecutar en desarrollo
npm run dev

# Ejecutar pruebas
npm test -- --runInBand

# Levantar PostgreSQL + API
docker compose up --build

# Levantar MongoDB simple + API
docker compose -f docker-compose-mongo.yml up --build

# Levantar clúster MongoDB + Redis + Elasticsearch + Nginx
docker compose -f docker-compose-mongo-cluster.yml up --build

# Inicializar sharding en Windows PowerShell
powershell -ExecutionPolicy Bypass -File scripts/init-mongo-cluster.ps1

# Construir imagen Docker de la API
docker build -t restaurant-reservations-api .
```

Anexo B. Resumen de rutas por archivo

Archivo	Rutas
authRoutes.js	POST /register, POST /login montadas bajo /auth
restaurantRoutes.js	GET /, POST / montadas bajo /restaurants
userRoutes.js	GET /me, PUT /:id, GET /, GET /:id, DELETE /:id montadas bajo /users
menuRoutes.js	GET /, GET /:id, POST /, PUT /:id, DELETE /:id montadas bajo /menus
dishRoutes.js	GET /, GET /:id, POST /, PUT /:id, DELETE /:id montadas bajo /menus/:menuId/dishes
reservationRoutes.js	POST /, DELETE /:id montadas bajo /reservations
orderRoutes.js	POST /, GET /:id montadas bajo /orders
searchRoutes.js	GET /, POST /reindex montadas bajo /search